Metro Ticket Generating System

ServiceNow-based project report, design, planning and testing pack

Updated 15 October 2026 | Academic prototype

Important: This is a prototype guide. It is not connected to a live metro operator, payment gateway, official fare authority, or gate validator. QR images are demo-only until backed by a trusted verification service.

1. Executive summary

The system proposes a ServiceNow-based workflow for requesting metro ticket details. A passenger selects origin, destination, travel date, and passenger count; server-side validation and configurable fare logic run; a booking record and ticket identifier are created; and a confirmation notification is sent.

The project package contains design and planning drafts, ServiceNow setup notes, a QR demo generator, and functional/performance/UAT test templates. Actual ServiceNow configuration and test execution remain the team's responsibility.

2. Problem and objectives

Passengers may experience queues or disconnected booking steps. The prototype aims to provide a guided booking request, consistent records, a configurable fare estimate, a privacy-safe QR payload, traceable status, and automated confirmation.

3. Scope and assumptions

In scope: catalog request, route validation, sample fare calculation, booking record, ticket identifier/QR payload, notifications, staff status lookup, and test planning. Out of scope: live operator APIs, payment/refunds, official ticket validity, gate hardware, and real-time service disruptions.

Demo fare assumption: INR 10 base plus INR 5 per station interval, multiplied by passenger count. This is not an official fare and must be replaced with operator-approved rules.

4. Architecture and data model

Passenger interface (Service Catalog / Employee Center) → server-side validation and fare service → Flow Designer → booking/ticket record → QR payload/ticket view → notification. A trusted verification endpoint is required for scanners.

Suggested tables: x_metro_station (station code/name/line/sequence/active), x_metro_fare_rule (approved fare configuration/effective dates), and x_metro_booking (requester, route, date, passenger count, fare, status, ticket ID, QR payload, notification status). Protect tables with role-based ACLs and audit key changes.

5. Functional requirements

FR-01 capture origin, destination, travel date, passenger count. FR-02 require two different active stations. FR-03 validate date and passenger-count bounds. FR-04 calculate fare on the server. FR-05 create a unique ticket/booking record. FR-06 generate QR payload without personal/payment data. FR-07 send confirmation or log failure. FR-08 allow authorized staff to inspect/update status. FR-09 maintain audit evidence.

6. Design phase outputs

Target segments: commuters, occasional passengers, tourists, and support staff. Proposed solution: a ServiceNow catalog request with server-side validation, configurable fare logic, Flow Designer orchestration, ticket record creation, QR payload generation, and notifications.

Architecture layers and assumptions are documented in design_phase/Metro_Ticketing_Project_Design.docx. The source design archive was mostly blank templates/examples, so the included project-specific design is a draft rather than user research evidence.

7. Planning phase outputs

Proposed 48-point backlog across five one-week sprints: Sprint 1 discovery/data model/ACLs/catalog draft (11); Sprint 2 catalog/validation/fare logic (11); Sprint 3 ticket record/QR/Flow Designer (11); Sprint 4 notifications/support/testing (10); Sprint 5 UAT/readiness (5).

Actual velocity and burndown are not reported because no completed sprint data was supplied. Use the planning document to update real completion and release dates.

8. Development and testing

Functional coverage includes valid booking, same-station rejection, inactive station, invalid date/count, fare tampering, unique IDs, QR privacy, notification success/failure, duplicate flow retry, ticket verification, ACLs, and accessibility.

Performance/reliability coverage includes booking latency, concurrent requests, flow duration, notification latency, recovery, and representative data volume. Set targets before execution; no response-time or performance result is fabricated.

UAT is pending. Run tests in a sub-production ServiceNow instance, record actual results and evidence, log defects, and obtain project manager/product owner sign-off.

9. Security, privacy and production readiness

Recompute fare server-side; use least-privilege ACLs; keep credentials out of source; validate inputs; do not embed personal/payment data in QR; use a trusted verification endpoint to check ticket status, travel window, and reuse policy. Apply retention and accessibility requirements.

Before production, obtain operator approval for fare policy, station/route data, payment, refunds, ticket validity, privacy, and security. Validate duplicate-trigger/retry behavior and monitoring.

10. Files included and source review

The source ZIPs are retained in source_templates/. Project-specific design, planning, and testing/UAT documents are in their respective folders. The development source archive's BI, Salesforce, AI/ML, and GenAI templates were not treated as relevant metrics for this ServiceNow workflow.

All tests are marked Not executed / Pending until actual runs are completed. This package is not evidence of deployment or successful acceptance testing.